

Competitive Programming

Section 32: Combinatorics Under Modulo

Competitive Programming Course

2026-07-10

Table of contents

1	Why counting needs an engine	3
2	Scope: when the factorial method is legal	4
3	Factorials and inverse factorials	5
3.1	Trace: build $C(6,2)$	5
4	Duplicate permutations	6
4.1	Trace: rearrange aabc	6
5	Stars and bars	7
5.1	Trace: edge cases	7
6	Choosing the preprocessing limit	8
7	C++ and Python notes	9
8	Common wrong approaches	10
9	Formative checks	11
10	Local judge workflow	12
11	Submission checklist	13
12	Exercises	14

1 Why counting needs an engine

Many counting statements reduce to a binomial coefficient, but the answer is requested modulo 1,000,000,007. The mathematical fraction $n!/(k!(n-k)!)$ cannot be evaluated with integer division after taking a remainder. This section builds one reusable engine for many queries.

Statement shape	Object to count
choose k positions/items from n	$C(n,k)$
rearrange a string with duplicate letters	$n! / \text{product}(\text{count!})$
give identical objects to distinct recipients, zero allowed	stars and bars
monotone grid path	choose positions of one move type

2 Scope: when the factorial method is legal

The local modulus is the prime $\text{MOD} = 1,000,000,007$, and every factorial index used by its problems is at most $2,000,000$. In particular, every index is strictly less than MOD .

This condition matters. If $i < \text{MOD}$, then $i!$ is not divisible by MOD , so it has a modular inverse. If an index reaches MOD , MOD divides $i!$ and this inverse-factorial method is no longer valid. Do not silently reuse it for $n \geq \text{MOD}$; that needs a later technique such as Lucas' theorem.

3 Factorials and inverse factorials

Precompute once up to the largest required index L:

```
fact[0] = 1
fact[i] = fact[i-1] * i mod MOD

invfact[L] = fact[L]^(MOD-2) mod MOD
invfact[i-1] = invfact[i] * i mod MOD
```

The first inverse uses Fermat's theorem from Section 29. The backward rule is valid because $i! = i * (i-1)!$; multiplying $\text{inverse}(i!)$ by i gives $\text{inverse}((i-1)!)$. It gives every inverse factorial with one exponentiation, not L exponentiations.

After preprocessing:

```
C(n,k) = fact[n] * invfact[k] * invfact[n-k] mod MOD
```

This is $O(L)$ preprocessing, $O(1)$ per query, and $O(L)$ memory.

3.1 Trace: build C(6,2)

Ordinarily, $C(6,2)=720/(2*24)=15$. In modular arithmetic, store:

```
fact[6] = 720
invfact[2] = inverse(2)
invfact[4] = inverse(24)
```

Then multiply the three residues. The product is the field version of the same division and equals 15. Also check the boundary cases before coding: $C(n,0)=C(n,n)=1$.

4 Duplicate permutations

Temporarily label equal objects. A string of length n then has $n!$ labelled permutations. If a letter appears c times, swapping just those labels does not change the visible string, so every visible arrangement was counted $c!$ times. Therefore:

```
distinct arrangements = n! / product_over_letters(count[letter]!)
```

4.1 Trace: rearrange aabc

There are $4!=24$ labelled arrangements. The two **a** labels can be swapped in each visible arrangement, so each appears twice in that count:

```
24 / 2! = 12
```

Letters absent from the string have count zero and contribute $0! = 1$; do not special-case them in code.

5 Stars and bars

To solve $x_1 + x_2 + \dots + x_n = s$ with $x_i \geq 0$, draw s stars and $n-1$ bars. The number of stars before/between/after bars is the allocation.

For three children and four candies:

```
**|*|*    means (2, 1, 1)
```

There are $s+n-1$ positions, and choosing the $n-1$ bar positions determines the distribution:

```
answer = C(s+n-1, n-1)
```

5.1 Trace: edge cases

For one child and ten candies, there is exactly one string of stars and no bars, so $C(10,0)=1$. For five children and zero candies, all four bars are adjacent, so $C(4,4)=1$. These are useful checks that the formula includes zero allocations correctly.

6 Choosing the preprocessing limit

This is a common source of memory-safe but wrong code.

Exercise	largest required factorial index
A. Choose Queries	maximum n
B. Rearrange Letters	$\text{len}(s)$
C. Distribute Candies	maximum $n+s-1$

For C, precomputing only through $\text{max}(n)$ fails as soon as candies are added. Read all queries first, find the maximum index, then allocate the two arrays.

7 C++ and Python notes

C++. Store values in `long long`; the product of two residues below $1e9+7$ fits in signed 64-bit, but not in `int`. A lambda for $C(n,k)$ should return zero if $k < 0$ or $k > n$, which is convenient in later formulas.

Python. `pow(value, MOD-2, MOD)` performs modular exponentiation directly. Python integers do not overflow, but lists of two million Python integers are still substantial; use one shared precomputation, not a table per query.

8 Common wrong approaches

Taking $n! \% \text{MOD}$, then performing ordinary integer division.
Using factorial inverses when a factorial index is at least MOD.
Computing one exponentiation for every inverse factorial.
Precomputing to $\max(n)$ for stars-and-bars instead of $\max(n+s-1)$.
Treating identical candies as distinct, or repeated letters as distinct.
Forgetting that $\text{fact}[0]$ and $\text{invfact}[0]$ are both 1.

9 Formative checks

1. Fill `fact[0..5]` and use the backward recurrence to recover `invfact[3]` from `invfact[5]`. State the algebra that justifies the step.
2. Count the rearrangements of `aabbc`. Expected answer: $5!/(2!*2!)=30$.
3. Draw the stars-and-bars strings for $x_1+x_2+x_3=2$; verify that there are $C(4,2)=6$ distributions.
4. For queries $(n,s)=(3,4)$ and $(100,1)$, identify the exact maximum factorial index before allocating any arrays. Expected value: 100.

10 Local judge workflow

Run the local solutions and randomized checks with:

```
CP_TARGET=solution python3 sections/32_combinatorics_under_modulo/check.py
```

The generators compare small cases with exact integer combinations and permutation enumeration. When a modular answer looks suspicious, first compare the same small case before applying $\% \text{ MOD}$.

11 Submission checklist

1. Is MOD prime and is every factorial index strictly below MOD?
2. What is the maximum factorial index required by every query?
3. Are fact[0] and invfact[0] both initialized to 1?
4. Does the story describe distinct or identical objects?
5. Did you test $k=0$, $k=n$, $s=0$, and $n=1$?

12 Exercises

Attempt A after implementing one `prepare(limit)` helper. Attempt B by reusing the same idea with character counts. Attempt C only after writing down why the limit is $n+s-1$, not merely n or s .